Theme: Social Media Profile Manager

Section – A

Q1. Explain the Django Request–Response Cycle and how it differs from a standard Python script execution.

Answer

The **Django Request–Response Cycle** is the process through which Django handles a user's request and returns an appropriate response. Whenever a user enters a website URL in a web browser, the browser sends an **HTTP Request** to the Django server. Django first checks the URL in the **urls.py** file to determine which view function should handle the request. The selected view processes the request, performs any required business logic, and may interact with the database using Django models. If data needs to be displayed, the view sends it to an HTML template, which generates the final web page. Finally, Django returns an **HTTP Response** to the browser.

In contrast, a standard Python script executes sequentially from top to bottom when the file is run. It does not wait for browser requests or communicate using HTTP. Python scripts are mainly used for calculations, file handling, automation, or console-based applications.

Example

Python Script

```
name = input("Enter your name: ")
print("Welcome", name)
```

Django View

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Django!")
```

Difference

Django	Python Script
Handles HTTP requests	Executes directly
Uses URLs, Views, Models	Runs line by line
Returns web pages	Prints console output
Suitable for web applications	Suitable for general programming

Conclusion

The Django Request–Response Cycle provides a structured approach to handling web requests, database operations, and dynamic web pages, whereas a standard Python script simply executes instructions sequentially without handling web communication.

Q2. Explain why Django Model Fields (CharField, IntegerField) are more robust for profile data than Python dynamic typing.

Answer

Django Model Fields provide a structured and reliable way to store data in a database. Fields such as **CharField**, **IntegerField**, **EmailField**, and **DateField** define the type of data that each column should store. This ensures data consistency, validation, and easier database management.

Python uses **dynamic typing**, meaning variables can change their data type during program execution. While this provides flexibility, it may also lead to errors if incorrect data types are assigned. For example, an age value may accidentally be stored as text instead of a number.

Django models prevent such problems because each field accepts only its specified data type. For instance, an IntegerField stores only integers, while a CharField stores only text up to a defined length. Django also performs validation before saving data, reducing the chances of invalid entries.

Example

```
from django.db import models

class Profile(models.Model):
    username = models.CharField(max_length=50)
    age = models.IntegerField()
```

Here, the **username** must be text, and **age** must be an integer.

Python Dynamic Typing Example

```
age = 25
age = "Twenty Five"
```

Although Python allows this, it may create errors in applications.

Advantages of Django Model Fields

- Data validation
- Better database integrity
- Prevents invalid data
- Easy database migration
- Supports relationships between models

Conclusion

Django Model Fields provide strong data validation and database consistency, making them much more suitable for storing user profile information than Python's dynamic typing.

Q3. Explain how Django Forms handle automated input validation for usernames and age ranges.

Answer

Django Forms simplify user input handling by providing built-in validation. Instead of manually checking every input, Django automatically validates data according to the field definitions. This reduces coding effort and improves application security.

For example, a username field can require a minimum and maximum number of characters, while an age field can accept only integer values within a specific range. If the entered data is invalid, Django displays meaningful error messages without crashing the application.

Developers can also create custom validation methods using the **clean()** function or **clean_fieldname()** methods. This allows additional checks such as ensuring that usernames are unique or that age is greater than 18.

Example

```
from django import forms

class RegisterForm(forms.Form):
    username = forms.CharField(min_length=4, max_length=20)
    age = forms.IntegerField(min_value=18, max_value=60)
```

Valid Input

```
Username: Meera123
Age: 22
```

Invalid Input

```
Username: Ab
Age: 70
```

Errors

- Username must contain at least 4 characters.
- Age must be between 18 and 60.

Advantages of Django Forms

- Automatic validation
- Better security
- Prevents invalid data
- Displays user-friendly error messages
- Reduces coding effort

Q4. Explain how to implement conditional logic in Django Templates to toggle account visibility.

Answer

Django Templates provide a simple way to display content dynamically using template tags. One of the most useful template tags is the **`{% if %}`** statement, which allows developers to show or hide content based on specific conditions. This feature is known as **conditional logic**.

For example, in a user profile page, an account may have a visibility status such as **Public** or **Private**. If the account is public, the template displays the user's profile information. If the account is private, the template hides the information and shows a message indicating that the profile is not available. This improves user privacy and enhances the user experience.

Conditional logic is processed on the server before the page is sent to the browser. Developers can also use **`elif`** and **`else`** blocks to handle multiple conditions. Django templates keep business logic separate from HTML, making the code cleaner and easier to maintain.

Example

View

```
def profile(request):
    return render(request, "profile.html", {
        "is_public": True
    })
```

Template

```
{% if is_public %}
<h2>User Profile</h2>
<p>Welcome to the public profile.</p>
{% else %}
<p>This account is private.</p>
{% endif %}
```

Output

If **`is_public = True`**

```
User Profile
Welcome to the public profile.
```

If **`is_public = False`**

```
This account is private.
```

Advantages

- Displays content dynamically.
- Improves privacy and security.
- Keeps HTML clean and organized.
- Easy to read and maintain.

Conclusion

Conditional logic in Django Templates allows developers to control what users can see based on specific conditions. It is commonly used for account visibility, login status, user roles, and permissions.

Q5. Explain the difference between iterating through a Python list and a Django QuerySet.

Answer

A **Python List** is a built-in data structure used to store multiple values in memory. It is suitable for handling data that already exists within a Python program. Developers can easily iterate through a list using a **for loop**.

A **Django QuerySet**, on the other hand, is a collection of database records returned by Django's Object Relational Mapper (ORM). Instead of storing data in memory, a QuerySet retrieves data from the database only when it is needed. This feature is known as **lazy evaluation**, which improves application performance.

Although both lists and QuerySets can be iterated using a loop, their purposes are different. A Python list is static unless modified manually, whereas a QuerySet always reflects the latest database records. QuerySets also support filtering, sorting, and searching using ORM methods.

Python List Example

```
students = ["Meera", "Rahul", "Asha"]

for student in students:
    print(student)
```

Django QuerySet Example

```
students = Student.objects.all()

for student in students:
    print(student.name)
```

Difference

Python List	Django QuerySet
Stores data in memory	Retrieves data from database
Static	Dynamic
Faster for small data	Efficient for large databases
No database connection	Connected with ORM

Conclusion

Python lists are ideal for temporary data storage, whereas Django QuerySets are designed for efficiently retrieving and managing database records in web applications.

Q6. Explain why the Django ORM is preferred over Python dictionaries for persistent profile storage.

Answer

The **Django Object Relational Mapper (ORM)** is a framework that allows developers to interact with a database using Python classes instead of writing SQL queries. It automatically converts Python objects into database records and vice versa. This makes database operations easier, safer, and more organized.

A **Python dictionary** stores data only in memory while the program is running. Once the program stops, the stored information is lost unless it is manually saved to a file. Therefore, dictionaries are not suitable for storing permanent user profile data.

Django ORM stores information in a relational database such as SQLite, MySQL, or PostgreSQL. The stored data remains available even after the application is closed or restarted. The ORM also provides built-in features such as validation, security against SQL injection, filtering, updating, deleting, and managing relationships between tables.

Python Dictionary Example

```
profile = {
    "username": "Meera",
    "age": 22
}

print(profile["username"])
```

Django ORM Example

```
class Profile(models.Model):
    username = models.CharField(max_length=50)
    age = models.IntegerField()
```

Saving data:

```
Profile.objects.create(
    username="Meera",
    age=22
)
```

Advantages of Django ORM

- Permanent data storage.
- Database security.
- Easy CRUD operations.
- Supports relationships.
- Automatic SQL generation.
- Better scalability.